Compterendu 3

groupe 4

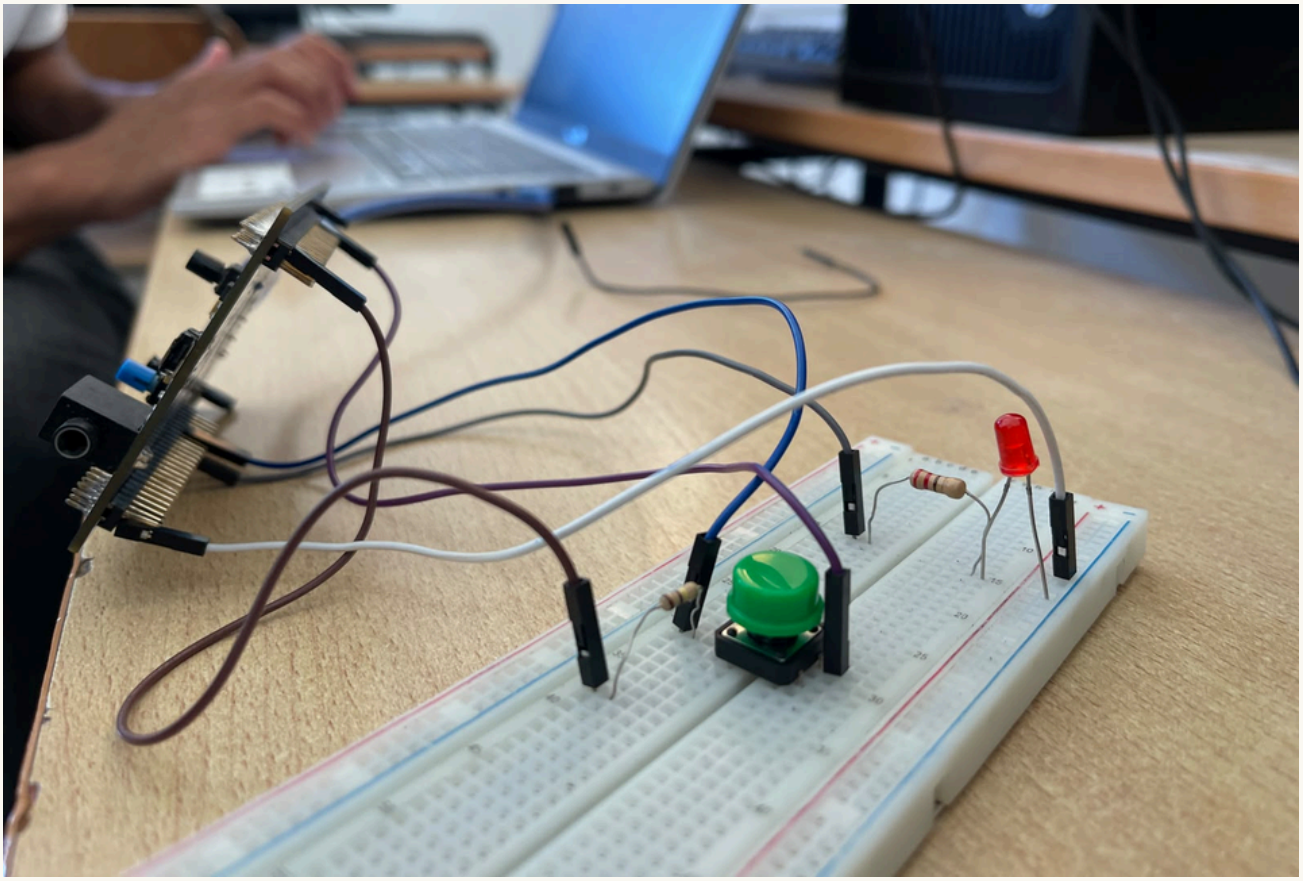
PROGRAMMATION CIBLE EMBARQUE

prof:

Ftirich Atef

étudiants:

Fadwa Bougoffa
Islem Guesmi
Mohamed wahib Elfahem



Ce premier TP a pour objectif de se familiariser avec la carte STM32 et de comprendre le fonctionnement des entrées et des sorties numériques (GPIO). Le travail consiste à commander une LED à l'aide d'un bouton poussoir : lors d'un premier appui sur le bouton, la LED s'allume, puis lors d'un deuxième appui, elle s'éteint. À travers cette manipulation, nous cherchons à comprendre le principe de lecture de l'état d'un bouton, la commande d'une sortie numérique et la programmation du microcontrôleur STM32. Ce TP nous permet également de mettre en pratique les notions théoriques étudiées en cours, depuis le câblage du circuit jusqu'à l'écriture et à l'exécution du programme.

1.1 etape n°1 : Cablage

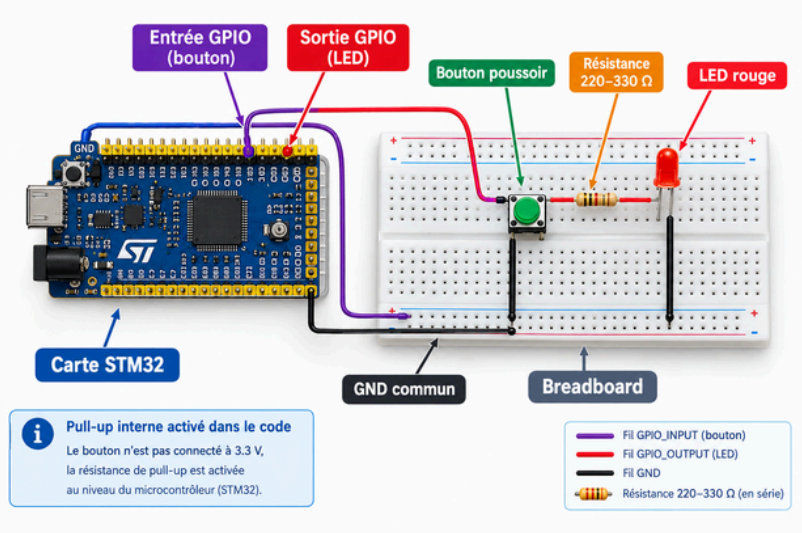
Dans cette première étape, nous avons réalisé le câblage du circuit électronique en utilisant

- une carte de développement STM32
- une plaque d'essai (breadboard)
- un bouton poussoir
- un LED
- des résistances
- des fils de connexion (Dupont).

Le bouton poussoir est utilisé comme entrée numérique pour détecter les appuis de l'utilisateur, tandis que la LED rouge représente la sortie numérique commandée par le microcontrôleur.

Nous avons placé les composants sur la breadboard, puis nous les avons reliés aux broches correspondantes de la carte STM32 à l'aide des fils de connexion. Une résistance a été associée à la LED afin de limiter le courant qui la traverse et de la protéger. Le câblage du bouton permet au microcontrôleur de détecter son état, tandis que celui de la LED permet de contrôler son allumage et son extinction.

1.2 figures de cablage



1. Connexion du bouton poussoir

- **Première borne du bouton Broche GPIO configurée en entrée (INPUT)**

Deuxième borne du bouton GND (masse) (inversion de pole)

- **Résistance de pull-up Pull-up interne activé dans le programme STM32**

Avec cette configuration, l'entrée GPIO est normalement à l'état logique 1. Lorsque le bouton est appuyé, elle passe à 0.

2. Connexion de la LED

- **GPIO de sortie Anode de la LED (patte longue) via une résistance de 220 à 330 Ω**

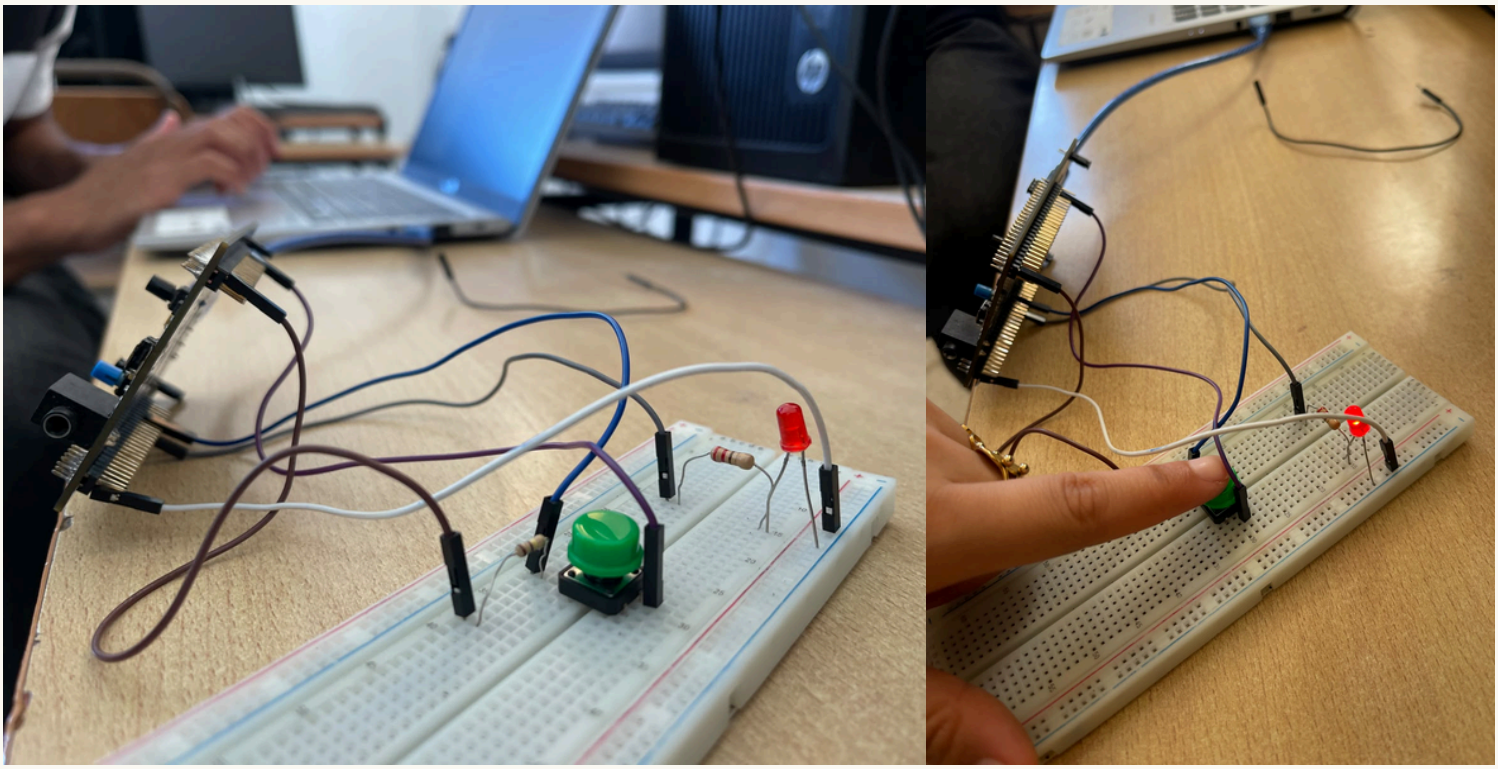
Cathode de la LED (patte courte) GND (masse)

La résistance limite le courant qui traverse la LED afin de la protéger. Lorsque la sortie GPIO est à 1, la LED s'allume ; lorsqu'elle est à 0, elle s'éteint.

3. Connexion de la carte STM32

- Relier le GPIO d'entrée au bouton poussoir.

- Relier le GPIO de sortie à la LED, à travers la résistance.



Câblage du bouton poussoir et de la LED rouge avec la carte STM32.

2.1 Deuxième étape : Parametrage de STM32

On a utilise le microcontrôleur STM32F407VGTx en boîtier LQFP100. les broches sélectionnées sont :

- PA0 : entrée GPIO pour le bouton poussoir (GPIO_Input).
- PA5 : sortie GPIO pour commander la LED (GPIO_Output).

1. Configuration des broches dans STM32CubeMX

GPIO Input

Configuration de PA0 — Bouton poussoir

1. on a sélectionne la broche PA0 dans la vue Pinout.
2. on a choisi GPIO_Input.
3. Dans les paramètres GPIO, on a sélectionne Pull-up si le bouton relie PA0 à GND lorsqu'on appuie dessus.
4. la vitesse par est choisi défaut, car il s'agit d'une entrée.

STM32CubeIDE GPIO

Configuration de PA5 — LED rouge

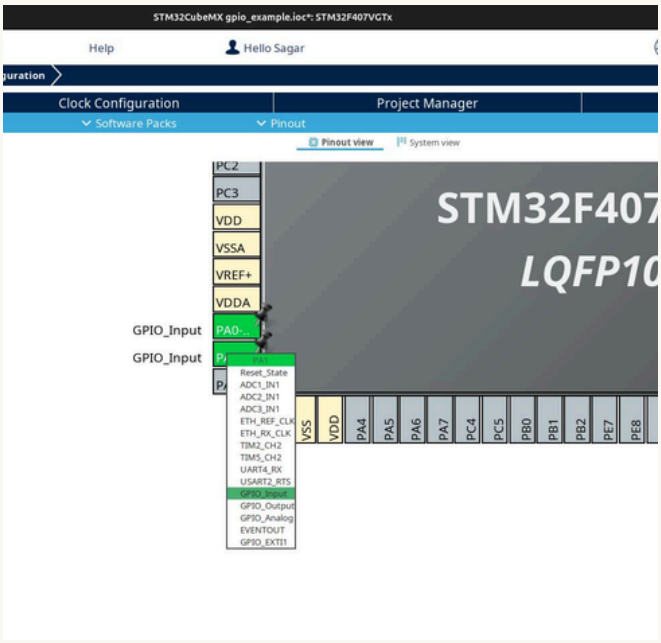
1. ona sélectionne la broche PA5.
2. on a choisi GPIO_Output.
3. ona a configure le mode de sortie en Output Push Pull.
4. ona choisi No pull-up and no pull-down.
5. on a régle la vitesse sur Low, suffisante pour commander une LED.

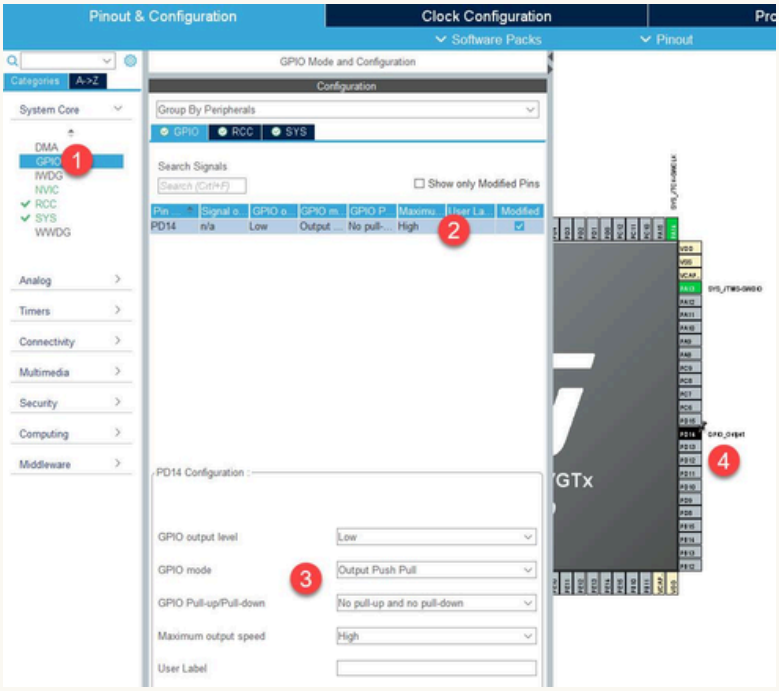
2. Configuration des paramètres GPIO

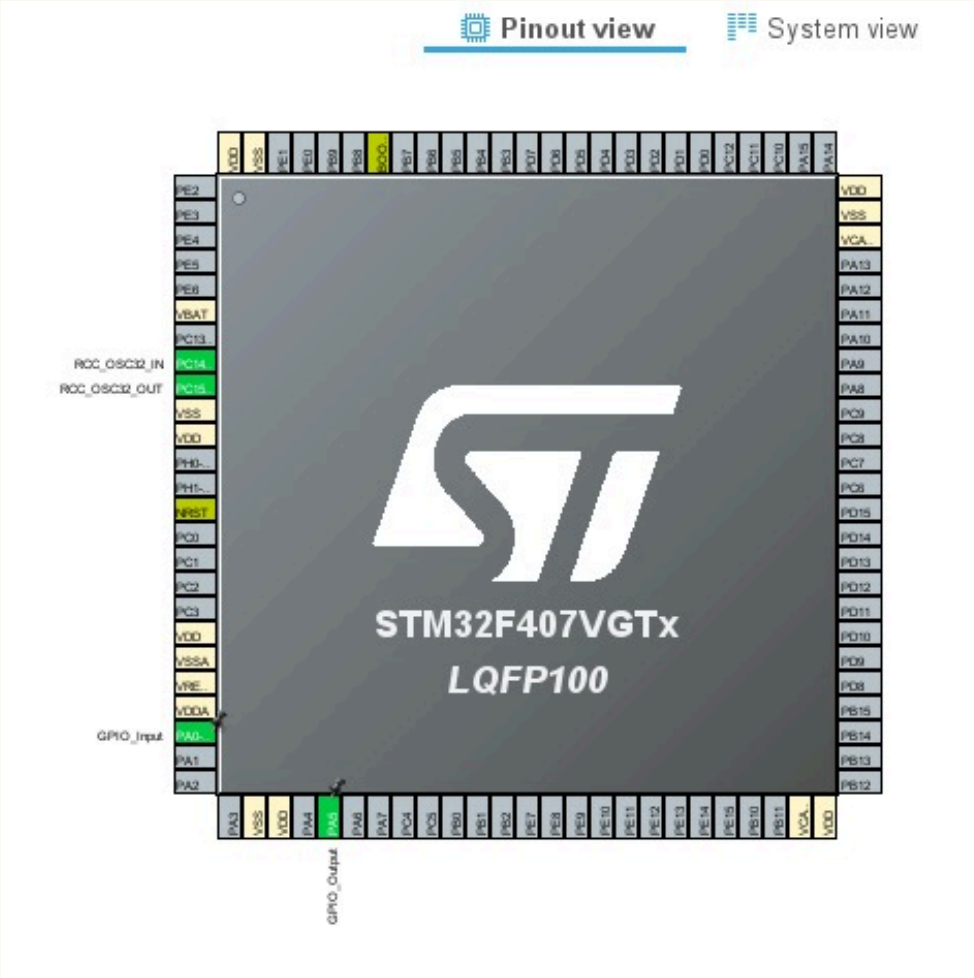
Dans l'onglet System Core → GPIO, on a défini les étiquettes suivantes :

PA0 — Bouton

PA5 — LED







Parametrage ou configuration de STM32

3.1 Troisieme étape : codage de carte STM32

main.c

```c
#include "stm32f4xx.h"

static void delay_ms(uint32_t ms)
{
    while (ms--)
        for (volatile uint32_t i = 0; i < 4000; i++) __NOP();
}

int main(void)
{
    /* Activer l'horloge GPIOA */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    (void)RCC->AHB1ENR;

    /* PA5 en sortie pour la LED */
    GPIOA->MODER &= ~(3U << (5 * 2));
    GPIOA->MODER |=  (1U << (5 * 2));

    /* PA0 en entrée pour le bouton */
    GPIOA->MODER &= ~(3U << (0 * 2));
    GPIOA->PUPDR &= ~(3U << (0 * 2));

    /* PA0 utilise une résistance externe de 10 kO en pull-down */

    while (1)
    {
        /* Détecter l'appui sur le bouton */
        if (GPIOA->IDR & GPIO_IDR_ID0)
```

```
24
25      while (1)
26      {
27          /* Détecter l'appui sur le bouton */
28          if (GPIOA->IDR & GPIO_IDR_ID0)
29          {
30              delay_ms(20);
31
32              if (GPIOA->IDR & GPIO_IDR_ID0)
33              {
34                  /* Inverser l'état de la LED */
35                  GPIOA->ODR ^= (1U << 5);
36
37                  /* Attendre le relâchement du bouton */
38                  while (GPIOA->IDR & GPIO_IDR_ID0);
39
40                  delay_ms(20);
41              }
42          }
43      }
44  }
```

3.2 explication du code

1. Inclusion de la bibliothèque

#include "stm32f4xx.h"

- **Cette instruction inclut le fichier d'en-tête de la famille STM32F4. Il permet d'accéder aux registres du microcontrôleur, comme ceux utilisés pour configurer les GPIO et l'horloge.**

2. Création de la fonction de temporisation

static void delay_ms(uint32_t ms)

- **static : limite l'utilisation de cette fonction au fichier courant.**
- **void : indique que la fonction ne renvoie aucune valeur.**
- **delay_ms : nom de la fonction, destinée à créer une temporisation.**
- **uint32_t ms : paramètre indiquant le nombre d'itérations de temporisation souhaitées, exprimé ici en millisecondes approximatives.**
- **while (ms--)**

La boucle while s'exécute tant que ms est différent de zéro. Après chaque vérification, la valeur de ms est décrémentée.

- **for (volatile uint32_t i = 0; i < 4000; i++)**

Cette boucle for réalise un certain nombre d'itérations afin de créer une attente. La variable i commence à zéro et augmente jusqu'à atteindre 4000.

Le mot-clé volatile empêche le compilateur de supprimer les accès à cette variable comme s'ils étaient inutiles.

- **__NOP();**

L'instruction __NOP() signifie No Operation. Elle exécute une instruction qui ne réalise aucune opération particulière, mais consomme un cycle d'exécution du processeur.

3. Fonction principale

int main(void)

La fonction main() est le point d'entrée du programme. Elle contient les instructions exécutées par le microcontrôleur après son démarrage.

4. Activation de l'horloge du port GPIOA

- **RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;**
- **Cette instruction active l'horloge du port GPIOA en positionnant le bit correspondant dans le registre AHB1ENR du contrôleur d'horloge RCC.**
- **L'activation de cette horloge est nécessaire pour pouvoir configurer et utiliser les broches PA0 et PA5.**

(void) RCC->AHB1ENR;

Cette lecture du registre permet de laisser un court délai après l'activation de l'horloge, afin que celle-ci soit prise en compte avant de poursuivre la configuration.

5. Configuration de PA5 en sortie pour la LED

- **GPIOA->MODER &= ~(3U << (5 * 2));**

Cette instruction efface les deux bits du registre MODER correspondant à PA5. Chaque broche GPIO possède deux bits permettant de sélectionner son mode de fonctionnement.

- **GPIOA->MODER |= (1U << (5 * 2));**

Cette instruction positionne les bits de PA5 sur 01, ce qui configure cette broche en sortie numérique.
La broche PA5 pourra ainsi commander l'allumage et l'extinction de la LED.

6. Configuration de PA0 en entrée pour le bouton

- **GPIOA->MODER &= ~(3U << (0 * 2));**

Cette instruction met à zéro les deux bits de configuration de PA0. La valeur 00 correspond au mode entrée numérique.

- **GPIOA->PUPDR &= ~(3U << (0 * 2));**

Cette instruction met à zéro les deux bits du registre PUPDR correspondant à PA0. Elle désactive les résistances internes de pull-up et de pull-down.
Lorsque le bouton relie PA0 à 3,3 V, l'entrée passe à l'état logique haut (1).

7. Boucle principale

- **while (1)**

Cette boucle infinie permet au microcontrôleur de surveiller continuellement l'état du bouton. Le programme ne s'arrête donc pas après un seul appui.

8. Détection de l'appui sur le bouton

if (GPIOA->IDR & GPIO_IDR_ID0)

GPIOA->IDR : registre qui permet de lire l'état logique des broches du port A.

- **GPIO_IDR_ID0 : masque correspondant au bit de PA0.**
- **& : opérateur logique de masquage bit à bit.**

La condition est vraie lorsque PA0 est à l'état logique haut (1), ce qui correspond à un bouton appuyé dans le montage décrit.

9. Première temporisation

- **delay_ms(20);**

Une temporisation d'environ 20 ms est ajoutée après la détection de l'appui. Elle sert à réduire les effets des rebonds mécaniques du bouton, qui peuvent provoquer plusieurs changements rapides de signal lors d'un seul appui.

10. Vérification de l'appui

- **if (GPIOA->IDR & GPIO_IDR_ID0)**

Après la temporisation, le programme vérifie à nouveau l'état de PA0. Si le bouton est toujours appuyé, le programme poursuit l'exécution. Cette seconde lecture permet d'éviter de réagir à certains rebonds du bouton.

11. Inversion de l'état de la LED

- **GPIOA->ODR ^= (1U << 5);**

Cette instruction inverse le bit 5 du registre ODR, qui commande l'état de sortie de PA5.
Si PA5 était à 0, il passe à 1 et la LED s'allume.
Si PA5 était à 1, il passe à 0 et la LED s'éteint.
L'opérateur ^= effectue un XOR entre la valeur actuelle et le masque du bit 5. C'est ce mécanisme qui permet d'alterner entre les deux états à chaque appui détecté.

12. Attente du relâchement du bouton

- **while (GPIOA->IDR & GPIO_IDR_ID0);**

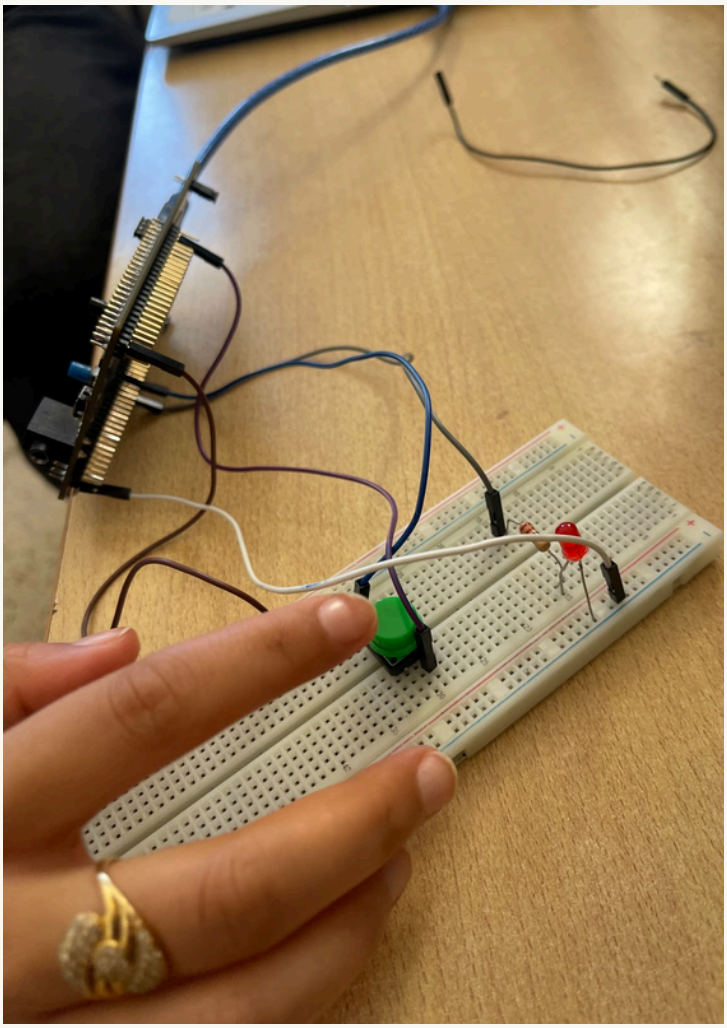
Cette boucle reste active tant que PA0 est à l'état logique haut. Elle empêche le programme de détecter plusieurs appuis successifs pendant que l'utilisateur maintient le bouton enfoncé.
Le point-virgule indique que le corps de la boucle est vide : le microcontrôleur attend simplement que le bouton soit relâché.

13. Deuxième temporisation

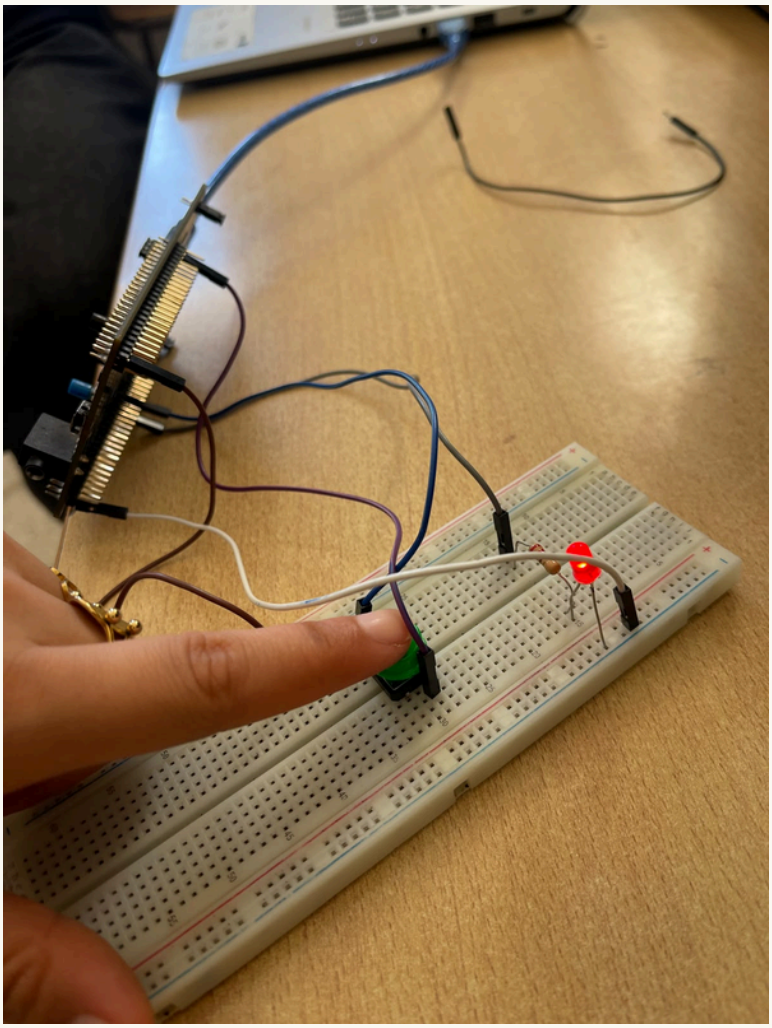
- **delay_ms(20);**

Une nouvelle temporisation est appliquée après le relâchement du bouton afin de limiter les effets des rebonds mécaniques lors du retour à l'état de repos.

4 resultat final : explication du code



- **bouton relache (avant cliquer)**



- **apres cliquer sur le bouton (allumer la lampe)**

4 resultat final

Ce TP nous a permis de nous familiariser avec la carte STM32F407VGTx et de mettre en pratique les notions de programmation des entrées-sorties numériques (GPIO). À travers la réalisation du câblage, le paramétrage des broches sous STM32CubeMX et l'écriture du programme en langage C, nous avons appris à détecter l'appui sur un bouton poussoir et à commander une LED rouge. L'utilisation des registres du microcontrôleur nous a également permis de mieux comprendre la configuration des GPIO, la lecture de l'état d'une entrée et la modification de l'état d'une sortie. Enfin, ce travail pratique nous a aidés à mieux comprendre l'interaction entre le matériel et le logiciel dans un système embarqué, tout en développant nos compétences en câblage électronique et en programmation des microcontrôleurs.